

Servomotoren der M17-Serie - DATENBLATT



Erschwingliche und einfache All-in-One-Bewegungssteuerung
Von der Ausbildung zur Innovation



Einführung

Die Servomotoren der M17-Serie sind All-in-One-Lösungen für die Bewegungssteuerung, die Motor, Motortreiber, Motion-Controller und Encoder in einem einzigen kompakten Gehäuse vereinen. Diese Servomotoren verfügen über eine RS-485-Kommunikationsschnittstelle, sodass mehrere Einheiten in Reihe geschaltet (Daisy-Chain) und über einen einzigen Anschlusspunkt gesteuert werden können. Die Serie ist in vier Modellen erhältlich - M17-60, M17-48, M17-40 und M17-34 - und bietet damit flexible Optionen für spezifische Drehmoment- und Größenanforderungen bei gleichbleibenden Regelungseigenschaften. Jeder Servomotor der M17-Serie bietet ausgereifte Steuerungsfunktionen, darunter mehrere Betriebsmodi, Selbstkalibrierung und integrierte Statusüberwachung über LED-Anzeigen. Die Motoren lassen sich einfach von jeder Plattform aus steuern, einschließlich Mac, PC, Raspberry Pi oder Arduino (es wird lediglich ein kostengünstiger RS485-Adapter benötigt), und eignen sich damit ideal für Ausbildungs- und Industrieanwendungen gleichermaßen. Die Serie unterstützt präzise Positionsregelung mit trapezförmigen Bewegungsprofilen, einen Closed-Loop-Regelmodus und eine umfassende Fehlerbehandlung. Mit standardisierten NEMA-17-Befestigungsmaßen, weitem Spannungsbereich (12-24V) und robustem Kommunikationsprotokoll bieten die Servomotoren der M17-Serie eine zuverlässige und flexible Lösung für Anwendungen, die eine präzise Bewegungssteuerung erfordern - von Robotik und CNC-Maschinen bis hin zu automatisierten Prüfeinrichtungen und wissenschaftlichen Instrumenten.

Senden Sie uns Ihr Feedback

Wenn Sie Fehler in diesem Dokument finden oder Fragen haben, teilen Sie uns dies bitte unter tutorial.gearotons.com/feedback mit. Wir freuen uns sehr über Ihre Vorschläge, wie wir das Produkt oder die Dokumentation verbessern können. Vielen Dank!

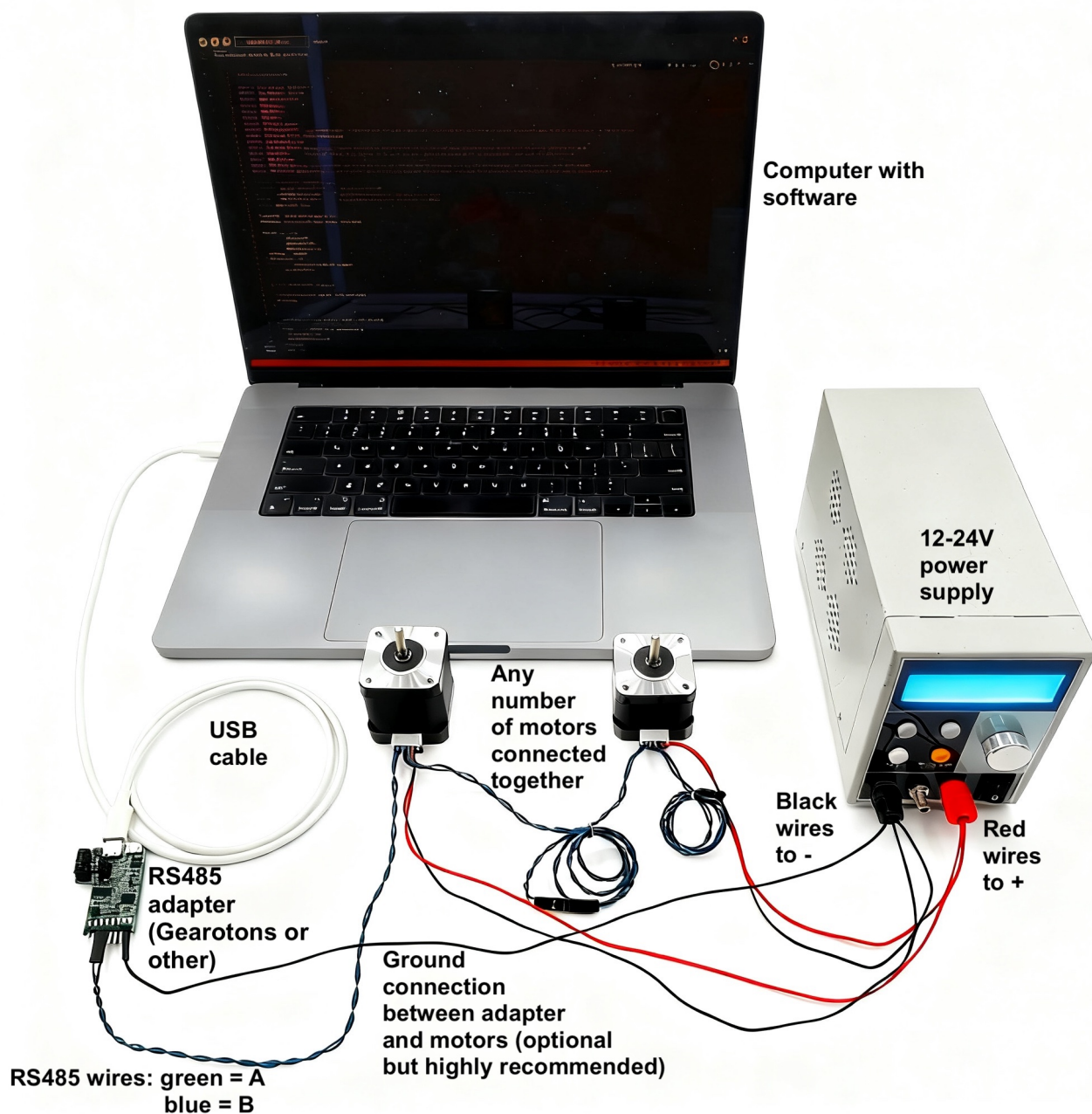


Besuchen Sie unsere Feedback-Seite

Wichtigste Merkmale

- Hoher Integrationsgrad: Motor, Motortreiber, Bewegungssteuerungssystem und Encoder in einem Gerät
- Steuerung beliebig vieler Motoren über eine RS-485-Kommunikationsschnittstelle mit nur einem einfachen Controller
- Steuerung über High-Level-Befehle wie "Enable mosfets" (MOSFETs aktivieren) und "Trapezoid move" (Trapezbewegung) - keine zeitkritische DIR/STEP-Ansteuerung erforderlich
- Kompakte Bauform, nahezu gleich groß wie ein NEMA-17-Schrittmotor mit denselben Spezifikationen (keine überstehenden Teile)
- Standardisierte NEMA-17-Befestigungsmaße
- Weiter Spannungsbereich (12-24V) für flexible Stromversorgungsoptionen
- Hochpräzise Closed-Loop-Regelung mit integriertem Encoder und PID-Regelkreis mit 32 kHz Regelfrequenz
- Deutlich energieeffizienter im Closed-Loop-Betrieb als ein herkömmlicher Schrittmotor
- Integrierter Überstrom-, Überspannungs- und Übertemperaturschutz
- Maximale Drehzahl bis zu 560 RPM
- Drehmoment-Gewichts-Verhältnis entspricht dem eines vergleichbaren Schrittmotors
- Kompatibel mit einer Vielzahl von Schnittstellen und Hardware, z. B. Raspberry Pi, Arduino, ESP32, Mac und PC
- Wir bemühen uns um hervorragende Dokumentation und Tutorials, damit Sie schnell einsatzbereit sind
- Wir stellen KI-freundliche Dokumentation bereit, falls Ihre bevorzugte KI die ganze Arbeit für Sie erledigen soll
- Geeignet für Robotik, CNC, Automatisierung, wissenschaftliche Instrumente, Prüfvorrichtungen, 3D-Drucker und vieles mehr
- In verschiedenen Größen erhältlich, damit Sie das passende Drehmoment und den passenden Preis für Ihre Anwendung finden

Anschlussdiagramm



Einheitensystem

Die Servomotoren der M17-Serie verwenden bestimmte interne Einheiten, um die mit der Bewegung verbundenen Berechnungen effizient (mit Ganzzahlarithmetik) durchführen zu können. Es liegt in der Verantwortung der steuernden Software, mehrere Maßeinheiten für die verschiedenen Größen zu unterstützen. Unsere Python- und Arduino-Bibliotheken führen Einheitenumrechnungen automatisch durch, sodass Sie mit Ihren bevorzugten Einheiten arbeiten können. Nachfolgend sind die unterstützten Einheiten für jede Größe aufgeführt:

Größe	Verfügbare Einheiten
Zeit	timesteps, seconds, milliseconds, minutes, microseconds
Position	shaft rotations, degrees, radians, encoder counts
Geschwindigkeit	rotations per second, rpm, degrees per second, radians per second, counts per second, counts per timestep
Beschleunigung	rotations per second squared, rpm per second, degrees per second squared, radians per second squared, counts per second squared, counts per timestep squared
Strom	internal current units, milliamps, amps
Spannung	millivolts, volts
Temperatur	celsius, fahrenheit, kelvin

Einstiegsanleitung

Um Ihnen den Einstieg mit Ihrem Servomotor der M17-Serie zu erleichtern, stellen wir eine umfassende Online-Anleitung bereit, die alles von der Ersteinrichtung bis zu fortgeschrittenen Protokollimplementierungen abdeckt. Diese Anleitung umfasst:

- Schritt-für-Schritt-Anleitung zur Einrichtung
- Ausführliche Dokumentation des Kommunikationsprotokolls
- Programmierbeispiele und Codeausschnitte
- Beschreibung der Fehlercodes
- Tipps zur Fehlerbehebung und bewährte Vorgehensweisen



Klicken Sie hier für die Einstiegsanleitung: tutorial.gearotons.com

Status-LEDs und Taster

Der Servomotor verfügt über zwei Status-LEDs (grün und rot). Die grüne LED blinkt langsam als Herzschlagsignal und schnell, um anzuzeigen, dass der Bootloader anstelle der Anwendung läuft. Die rote LED leuchtet kurz auf, um Kommunikation auf dem Bus anzuzeigen, und signalisiert fatale Fehlercodes durch eine bestimmte Anzahl von Blinkimpulsen.

Der Servomotor verfügt über zwei Taster mit den Bezeichnungen "Reset" und "Test". Der Reset-Taster setzt den internen Mikrocontroller zurück, und alle Zustände kehren zu den Standardwerten zurück. Der Test-Taster lässt den Motor drehen. Drücken Sie kurz, damit er in die eine Richtung dreht; drücken Sie länger als 0,3 Sekunden und lassen Sie los, damit er in die andere Richtung dreht. Halten Sie den Taster mindestens 2 Sekunden gedrückt und lassen Sie los, damit der Motor in den Closed-Loop-Modus wechselt. Halten Sie ihn länger als 15 Sekunden gedrückt und lassen Sie los, damit der Motor eine Selbstkalibrierung durchführt. Beachten Sie, dass sich der Motor während der Kalibrierung dreht und sich frei drehen können muss, damit die Kalibrierung erfolgreich ist. Entfernen Sie daher vor diesem Vorgang alle Lasten.

Überblick über das Kommunikationsprotokoll

Die Servomotoren der M17-Serie kommunizieren über RS-485, sodass mehrere Motoren in Reihe geschaltet (Daisy-Chain) werden können. Jeder Motor besitzt eine werkseitig zugewiesene, eindeutige 64-Bit-Adresse, über die er auf einem RS-485-Bus individuell angesprochen werden kann. Jedem Motor kann außerdem ein eindeutiger 1-Byte-Alias (0-251) zugewiesen werden, um Bytes im Kommunikationspaket zu sparen und dennoch eine vergleichbare individuelle Steuerung zu ermöglichen. Es gibt zudem einen Broadcast-Alias mit dem Wert 255. Das Protokoll unterstützt einen umfassenden Befehlssatz für Bewegungssteuerung, Konfiguration und Statusüberwachung. Firmware-Updates über die RS-485-Schnittstelle werden unterstützt. Die Baudrate der Kommunikation ist fest auf 230400 eingestellt.

Zusammenfassung der Befehlsreferenz

Die aktuelle, verbindliche Referenz aller verfügbaren Befehle finden Sie in diesem Dokument.



https://github.com/tomrodinger/servomotor/blob/main/python_programs/servomotor/motor_commands.json

Sie können auch diesen Befehl ausführen:

```
pip3 install servomotor # run this just once to install the library and programs
servomotor_command -c
```

Dadurch werden die in der Datei motor_commands.json enthaltenen Informationen übersichtlich ausgegeben, zusammen mit Hinweisen zum Senden von Befehlen an den Motor über die Kommandozeile.

Grundsteuerung

Befehl	Beschreibung
Disable MOSFETs	Deaktiviert die MOSFETs (beachten Sie, dass die MOSFETs nach dem Einschalten standardmäßig deaktiviert sind).
Enable MOSFETs	Aktiviert die MOSFETs.
Reset time	Setzt die absolute Zeit auf null zurück (rufen Sie diesen Befehl zuerst auf, bevor Sie Bewegungsbefehle senden).
Emergency stop	Führt einen Nothalt aus (stoppt alle Bewegungen, deaktiviert die MOSFETs und leert die Warteschlange).
Zero position	Macht die aktuelle Position zur Nullposition (dem Ursprung).
System reset	Setzt das System zurück und startet den Bootloader. Der Motor wird sofort zurückgesetzt und wechselt in den Bootloader. Wird innerhalb kurzer Zeit kein Befehl gesendet, verlässt der Motor den Bootloader und führt die Anwendung von Beginn an aus.

Bewegungssteuerung

Befehl	Beschreibung
Trapezoid move	Fährt sofort mit der aktuell eingestellten Geschwindigkeit zur angegebenen Position (die Geschwindigkeit wird über einen separaten Befehl eingestellt).
Go to position	Fährt innerhalb der angegebenen Zeit zur angegebenen Position. Beschleunigung und Abbremsung werden angewendet, um die Bewegung gleichmäßig zu gestalten.
Homing	Führt eine Referenzfahrt des Motors durch (d. h. der Motor fährt bis zum Anschlag und stoppt dann sofort).
Go to closed loop	Wechselt in den Closed-Loop-Positionsregelmodus.
Move with acceleration	Dreht den Motor mit der angegebenen Beschleunigung.
Move with velocity	Dreht den Motor mit der angegebenen Geschwindigkeit.
Multimove	Setzt mehrere Bewegungen zusammen, die nacheinander ausgeführt werden. Beachten Sie: Wenn die Warteschlange nach Ausführung aller Bewegungen leer ist und der Motor nicht stillsteht, wird ein fataler Fehler ausgelöst.

Konfiguration

Befehl	Beschreibung
Set maximum velocity	Stellt die maximale Geschwindigkeit ein (wird derzeit nicht verwendet).
Set maximum acceleration	Stellt die maximale Beschleunigung ein.
Start calibration	Startet eine Kalibrierung, die die Mittelwerte der Hall-Sensoren ermittelt und prüft, ob diese korrekt funktionieren.
Set maximum motor current	Stellt den maximalen Motorstrom und den maximalen Rekuperationsstrom ein. Die Werte werden sofort wirksam, aber nicht im nichtflüchtigen Speicher abgelegt; nach einem Reset gelten wieder die Firmware-Standardwerte.
Set safety limits	Stellt die Sicherheitsgrenzen ein (um zu verhindern, dass die Bewegung die festgelegten Grenzen überschreitet).
Test mode	Stellt einen bestimmten Testmodus ein oder löst ihn aus. Dies ist derzeit nur teilweise dokumentiert. Verwenden Sie diesen Befehl nur, wenn Sie als Entwickler an Testfällen arbeiten. Wichtige Modi: 0 = alle Testmodi löschen; 1-9 = Motor-Testmodi; 10-13 = LED-Testmodi (blockieren das Gerät); 14-73 = fatale Fehler 0-59 auslösen; 74 = Schwellwert des Überspannungsschutzes auf 22 V setzen (Produktionstest; sollte bei einer 24-V-Versorgung auslösen); 75 = Schwellwert des Überspannungsschutzes auf 26 V setzen (Produktionstest; sollte bei einer 24-V-Versorgung nicht auslösen); 76 = Übertemperaturabschaltung auf etwa 90 Grad C anheben (Produktionstest der thermischen Reserve; die Standardabschaltung liegt bei etwa 80 Grad C). Ein System-Reset stellt die Standardschwellwerte für Überspannung und Übertemperatur wieder her.
Set PID constants	Stellt die PID-Konstanten für den Regelkreis ein, der die Bewegungstrajektorie einzuhalten versucht.
Set max allowable position deviation	Stellt die Anzahl der Mikroschritte ein, um die die tatsächliche Motorposition (gemessen durch die Hall-Sensoren) von der Sollposition abweichen darf. Bei Überschreitung wird ein fataler Fehler ausgelöst.
CRC32 control	Aktiviert oder deaktiviert die CRC32-Prüfung für Befehle.

Status und Überwachung

Befehl	Beschreibung
Get current time	Liest die aktuelle absolute Zeit aus.
Get n queued items	Liest die Anzahl der Einträge in der Bewegungswarteschlange aus (wird diese zu groß, stellen Sie keine weiteren Bewegungsbefehle in die Warteschlange).
Get hall sensor position	Liest die von den Hall-Sensoren gemessene Position aus (dies sollte die tatsächliche Position des Motors sein; wenn alles in Ordnung ist, entspricht sie in etwa der Sollposition).
Get status	Liest den Status des Motors aus.
Control hall sensor statistics	Schaltet die Erfassung der Hall-Sensor-Statistiken ein oder aus und setzt die Statistiken zurück.
Get hall sensor statistics	Liest die von den Hall-Sensoren erfassten Statistiken aus. Dies ist nützlich, um den Zustand der Hall-Sensoren und das Rauschen im System zu prüfen.
Get position	Liest die aktuelle Sollposition aus (die geringfügig von der tatsächlichen, durch die Hall-Sensoren gemessenen Position abweichen kann).
Get comprehensive position	Liest die Sollposition des Motors, die Hall-Sensor-Position und die Position des externen Encoders in einem Zugriff aus.
Get supply voltage	Liest die gemessene Versorgungsspannung aus.
Get max PID error	Liest die minimalen und maximalen Fehlerwerte aus, die seit dem letzten Auslesen im PID-Regelkreis beobachtet wurden.
Get temperature	Liest die gemessene Temperatur des Motors aus.
Get debug values	Liest Debug-Werte aus, einschließlich Motorsteuerungsparametern, Profiler-Zeiten, Hall-Sensor-Daten und weiteren Diagnoseinformationen.
Get communication statistics	Liest den CRC32-Fehlerzähler aus und setzt ihn optional zurück.

Geräteverwaltung

Befehl	Beschreibung
Time sync	Sendet die Master-Zeit an den Motor, damit dieser seine eigene Uhr synchronisieren kann (führen Sie dies 10-mal pro Sekunde aus).
Get product specs	Liest die Aktualisierungsfrequenz aus (den Kehrwert des Zeitschritts).
Detect devices	Erkennt alle an der RS485-Schnittstelle angeschlossenen Geräte. Die Geräte melden sich zu einem zufälligen Zeitpunkt innerhalb einer Sekunde. Eine Kollision ist möglich, aber unwahrscheinlich. Sie können den Vorgang wiederholen, wenn Sie eine Kollision vermuten (zum Beispiel, wenn Geräte angeschlossen sind, aber nicht innerhalb von ein bis zwei Sekunden erkannt wurden).
Set device alias	Stellt den Geräte-Alias ein.
Get product info	Liest die Produktinformationen aus.
Firmware upgrade	Aktualisiert den Flash-Speicher des Servomotors. Bevor Sie einen Firmware-Upgrade-Befehl senden, müssen Sie einige Berechnungen durchführen, wie in den Beispielen gezeigt.
Get product description	Liest die Produktbeschreibung aus.
Get firmware version	Liest die Firmware-Version oder die Bootloader-Version aus, je nachdem, in welchem Modus sich das Gerät befindet. Dieser Befehl gibt außerdem die Statusbits zurück, wobei das niederwertigste Bit angibt, ob sich das Gerät derzeit im Bootloader (=1) oder in der Haupt-Firmware (=0) befindet.
Ping	Sendet eine Nutzlast mit beliebigen Daten; das Gerät antwortet mit denselben Daten.
Vibrate	Veranlasst den Motor, die Spannung schnell zu variieren und dadurch zu vibrieren (oder das Vibrieren zu beenden).
Identify	Identifizieren Sie Ihren Motor, indem Sie diesen Befehl senden. Die grüne LED des Motors blinkt 3 Sekunden lang schnell.

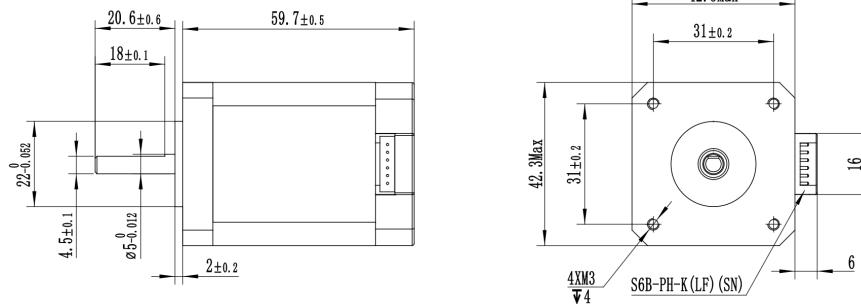
Sonstiges

Befehl	Beschreibung
Capture hall sensor data	Startet das Senden von Hall-Sensor-Daten (in Entwicklung; senden Sie diesen Befehl nicht).
Read multipurpose buffer	Liest den Inhalt des Mehrzweckpuffers aus (der Puffer wird für Daten verwendet, die während der Kalibrierung, beim Wechsel in den Closed-Loop-Modus und bei der Erfassung von Hall-Sensor-Daten erzeugt werden).

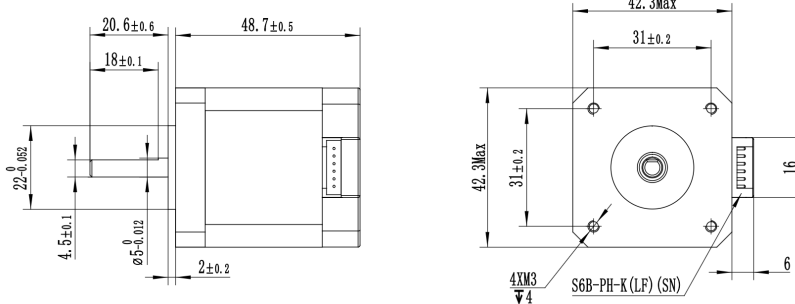
Mechanische Spezifikationen

Parameter	M17-60	M17-48	M17-40	M17-34
Abmessungen (LxB)	42.2x42.2 mm	42.2x42.2 mm	42.2x42.2 mm	42.2x42.2 mm
Höhe	59.7 mm	48.7 mm	40.1 mm	33.5 mm
Wellenlänge	20.6 mm	20.6 mm	20.6 mm	20.6 mm
Gewicht	470g	360g	285g	210g
Schutzart	IP20	IP20	IP20	IP20

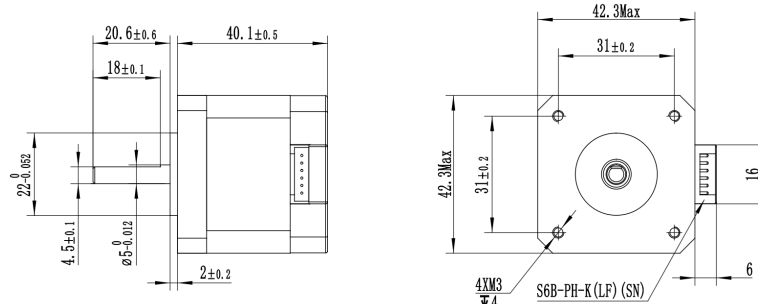
M17-60 Model



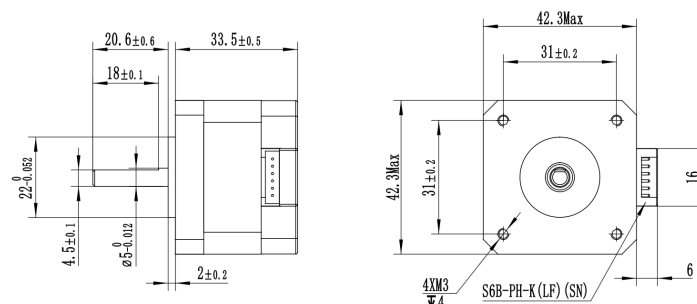
M17-48 Model



M17-40 Model



M17-34 Model



Technische Spezifikationen

Parameter	M17-60	M17-48	M17-40	M17-34
Betriebsspannung	12-24V	12-24V	12-24V	12-24V
Nenndrehmoment	0.65 N.m	0.55 N.m	0.42 N.m	0.28 N.m
Maximale Drehzahl	560 RPM	560 RPM	560 RPM	560 RPM
Maximaler Strom	1.1A	1.1A	1.1A	1.0A
Nennleistung	26.4W	26.4W	26.4W	24.0W

Betriebsbedingungen

Parameter	Spezifikation
Betriebstemperatur	0C bis +80C
Lagertemperatur	-20C bis +60C
Luftfeuchtigkeitsbereich	20% bis 80% RH (nicht kondensierend)
Einsatzumgebung	Nur für den Innenbereich

Python-Bibliothek

Unsere Python-Bibliothek bietet eine umfassende Schnittstelle zur Steuerung der Servomotoren der M17-Serie. Die Bibliothek übernimmt alle Low-Level-Kommunikationsprotokolle und Einheitenumrechnungen und erleichtert so die Integration der Servomotorsteuerung in Ihre Python-Projekte.

Installation

Installieren Sie die servomotor-Bibliothek mit pip:

```
pip3 install servomotor
```

Beispiel: Grundlegende Motorsteuerung

Das folgende Beispiel zeigt, wie Sie eine Verbindung zu einem Servomotor herstellen, die MOSFETs aktivieren und eine Trapezbewegung ausführen:

```
#!/usr/bin/env python3
"""
Minimal trapezoid move: rotate 1 turn in 1 second.
Edit ALIAS below if needed. Uses rotations and seconds.
"""
import time, servomotor
from servomotor import communication

# Hard-coded settings for a minimal demo
ALIAS = 'X' # Device alias, change if needed
SERIAL_PORT = "/dev/tty.usbserial-110" # Serial device path; change if needed (e.g., "COM3" on
# Windows)
DISPLACEMENT_ROTATIONS = 1.0 # 1 rotation
DURATION_SECONDS = 1.0 # 1 second
DELAY_MARGIN = 0.10 # +10% wait margin because the motor's clock is not
# perfectly accurate

communication.serial_port = SERIAL_PORT # if you comment this out then the program
# should prompt you for the serial port or it will use
# the last used port from a file

servomotor.open_serial_port()

m = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations", verbose=0)
m.enable_mosfets()
m.trapezoid_move(DISPLACEMENT_ROTATIONS, DURATION_SECONDS)
time.sleep(DURATION_SECONDS * (1.0 + DELAY_MARGIN))
m.disable_mosfets()

servomotor.close_serial_port()
```

Wichtigste Merkmale

- Unterstützung aller Motorbefehle
- Automatische Einheitenumrechnung für Zeit, Position, Geschwindigkeit, Beschleunigung, Strom, Spannung und Temperatur
- Fehlerbehandlung und Statusüberwachung
- Unterstützung mehrerer Motoren am selben Bus
- Einsatzbereit mit nur wenigen Zeilen Code

Dokumentation

Die vollständige API-Dokumentation und weitere Beispiele finden Sie unter:
https://github.com/tomrodinger/servomotor/tree/main/python_programs

Arduino-Bibliothek

Die Arduino-Bibliothek für die Servomotoren der M17-Serie bietet eine einfach zu bedienende Schnittstelle zur Steuerung der Motoren von Arduino-Boards aus. Die Bibliothek unterstützt serielle Hardware-Kommunikation und enthält alle wesentlichen Funktionen zur Motorsteuerung.

Installation

Installieren Sie die Bibliothek über den Arduino-Bibliotheksverwalter:

- 1. Öffnen Sie die Arduino IDE
- 2. Gehen Sie zu Sketch → Bibliothek einbinden → Bibliotheken verwalten
- 3. Suchen Sie nach "Servomotor"
- 4. Suchen Sie die Bibliothek mit dem Namen "Servomotor by Gearotons" und klicken Sie auf Installieren



Manuelle Installation

Alternativ können Sie als erfahrener Anwender den Code der Arduino-Bibliothek hier herunterladen:
https://github.com/tomrodinger/Servomotor_Arduino_Library

Beispiel: Grundlegende Motorsteuerung

Dieses Beispiel zeigt, wie Sie einen Motor initialisieren, die MOSFETs aktivieren und eine Trapezbewegung ausführen:

```

// Minimal Arduino example: Trapezoid move using built-in unit conversions
// Goal: spin the motor exactly 1 rotation in 1 second, then stop.
// Sequence:
// enable MOSFETs -> trapezoidMove(1.0 rotations, 1.0 seconds) -> wait 1.1s -> disable MOSFETs.
//
// Notes:
// - This uses the library's unit conversion (no raw counts/timesteps).
// - Configure Serial1 pins for your board (ESP32 example pins below).
// - Motor is created AFTER Serial1.begin(...) so hardware UART pins are set first.

#include <Servomotor.h>

#define ALIAS 'X' // Device alias
#define BAUD 230400 // RS485 UART baud rate
#define DISPLACEMENT_ROTATIONS 1.0f // 1 rotation
#define DURATION_SECONDS 1.0f // 1 second
#define TOLERANCE_PERCENT 10 // +10% wait margin because the motor's clock is not
// perfectly accurate
#define WAIT_MS ((unsigned long)(DURATION_SECONDS * 1000.0f * (100 + TOLERANCE_PERCENT) / 100))

// Example RS485 pins for ESP32 DevKit (change as needed for your board)
#if defined(ESP32)
#define RS485_TXD 4 // TX pin to RS485 transceiver
#define RS485_RXD 5 // RX pin from RS485 transceiver
#endif

void setup() {
  Serial.begin(115200); // Console serial for debugging

  // Create the motor; serial port opens on first instantiation.
  #if defined(ESP32)
    Servomotor motor(ALIAS, Serial1, RS485_RXD, RS485_TXD);
  #else
    Servomotor motor(ALIAS, Serial1);
  #endif

  // Use units: rotations for position, seconds for time
  motor.setPositionUnit(PositionUnit::SHAFT_ROTATIONS);
  motor.setTimeUnit(TimeUnit::SECONDS);

  motor.enableMosfets();
  motor.trapezoidMove(DISPLACEMENT_ROTATIONS, DURATION_SECONDS);
  delay(WAIT_MS);
  motor.disableMosfets();
}

void loop() {
}

```

Unsere Mission

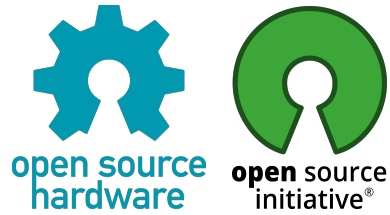
Wir bei Gearotons sind überzeugt, dass die Zukunft denjenigen gehört, die künstliche Intelligenz und Automatisierungssysteme verstehen und mit ihnen zusammenarbeiten können. Unsere Mission ist es, die nächste Generation von Innovatoren zu stärken, indem wir fortschrittliche Bewegungssteuerungstechnologie für Maker, Lehrende und Ingenieure weltweit zugänglich, verständlich und erschwinglich machen. Wir setzen uns dafür ein, die Barrieren abzubauen, die Studierende und Hobbyanwender traditionell von professionellen Automatisierungswerkzeugen getrennt haben. Mit integrierten Lösungen, die Komplexität beseitigen, ohne auf Leistungsfähigkeit zu verzichten, ermöglichen wir praxisnahe Lernerfahrungen, die junge Menschen auf eine Welt vorbereiten, in der menschliche Kreativität und maschinelle Präzision harmonisch zusammenwirken. Unsere Grundüberzeugung ist, dass jeder Studierende die Möglichkeit haben sollte, mit den Bausteinen der modernen Automatisierung zu experimentieren - von Robotik und CNC-Systemen bis hin zu wissenschaftlichen Instrumenten. Durch unsere Bildungspartnerschaften und unseren Open-Source-Ansatz fördern wir eine Generation, die Technologie nicht nur nutzt, sondern sie wirklich versteht und gestaltet. Gemeinsam schaffen wir das Fundament für Innovatoren, die die großartige Zukunft prägen werden.

Open Source

Wir glauben daran, die Welt durch Technologie besser zu machen. Software, Firmware und PCB-Designdateien sind hier verfügbar:



<https://github.com/tomrodinger/servomotor>



Datenblatt-Version: 1.9 Veröffentlichungsdatum: Jun. 29, 2026

(c) 2026 Alle Spezifikationen können ohne Vorankündigung geändert werden.